

МРС-62 — Добавить

RefreshProductCatalogUseCase

Summary

Подключить существующий механизм поиска товарных URL к постоянному каталогу `product_catalog`.

В рамках задачи необходимо реализовать отдельный application use case:

RefreshProductCatalogUseCase

Use case должен:

1. запустить существующий discovery товаров;
2. получить найденные URL;
3. преобразовать их в элементы каталога;
4. выполнить массовый upsert через `IProductCatalogRepository`;
5. создать и корректно завершить отдельный `crawler_run`;
6. вернуть структурированный результат обновления каталога;
7. не запускать сбор цен;
8. не использовать `price_collect_queue`;
9. не изменять существующий `RunCrawlerUseCase`.

После завершения задачи проект должен уметь отдельно обновлять постоянный каталог товаров, не выполняя извлечение карточек и цен.

Context

Репозиторий:

<https://github.com/bodomus/PriceCrawler>

Базовая ветка:

МРС-61

Создать отдельную рабочую ветку:

Codex/МРС-62

Dependency

Задача зависит от завершённой MPC-61.

До начала реализации убедиться, что в базовой ветке существуют:

- таблица `product_catalog`;
- `ProductCatalogItem`;
- `ProductCatalogUpsertItem`;
- `ProductCatalogUpsertResult`;
- `IProductCatalogRepository`;
- `PgProductCatalogRepository`;
- batch upsert routine для `product_catalog`;
- DI-регистрация repository.

Если перечисленных компонентов нет, не создавать альтернативную реализацию каталога внутри MPC-62. Зафиксировать блокирующую зависимость в отчёте.

Current behavior

Существующий crawler flow объединяет несколько операций:

```
Discovery
→ фильтрация URL
→ создание crawler_run
→ enqueue
→ извлечение карточек
→ сохранение product / price_snapshot
```

Постоянный каталог из MPC-61 пока не подключён к runtime.

Существующий `RunCrawlerUseCase` не должен быть изменён в рамках MPC-62.

Goal

Добавить самостоятельный сценарий:

```
RefreshProductCatalogUseCase
↓
IProductUrlDiscoveryService
↓
ProductCatalogUpsertItem[]
↓
IProductCatalogRepository.UpsertDiscoveredAsync
```

↓
product_catalog

Этот сценарий должен использовать уже реализованные:

- discovery strategy factory;
- CategorySeeds ;
- ProductUrlDiscoveryService ;
- ProductUrlFilter ;
- нормализацию URL;
- crawler run repository;
- постоянный product catalog repository.

Required architecture

Соблюдать текущую архитектуру:

```
Domain
Application
Infrastructure
Worker
Web
Tests
```

Domain

Domain должен содержать только:

- модели;
- value objects;
- repository interfaces;
- enums;
- domain contracts.

Domain не должен зависеть от:

- Application;
- Infrastructure;
- PostgreSQL;
- Npgsql;
- EF Core;
- Worker;
- Web.

Application

RefreshProductCatalogUseCase должен находиться в Application.

Application может зависеть от:

- `IProductUrlDiscoveryService`;
- `IProductCatalogRepository`;
- `ICrawlerRunRepository`;
- Application models;
- Domain models и interfaces;
- logging;
- options, только если это действительно необходимо.

Application не должен содержать SQL.

Infrastructure

В рамках задачи Infrastructure меняется только при необходимости регистрации уже существующих реализаций или исправления несовместимости DI.

Не переносить orchestration logic в Infrastructure.

Worker и Web

Worker или Web могут вызывать use case, но не должны содержать его бизнес-логику.

Scope

1. Добавить интерфейс use case

Добавить application contract:

```
public interface IRefreshProductCatalogUseCase
{
    Task<RefreshProductCatalogResult> ExecuteAsync(
        CancellationToken ct);
}
```

Название метода допускается:

RunAsync

или:

ExecuteAsync

Выбрать один вариант и использовать последовательно.

Предпочтительный вариант:

ExecuteAsync

2. Добавить модель результата

Добавить immutable result model:

```
public sealed record RefreshProductCatalogResult(  
    long RunId,  
    string Status,  
    string Source,  
    int DiscoveredCount,  
    int AcceptedCount,  
    int InsertedCount,  
    int UpdatedCount,  
    int SkippedCount,  
    string? ErrorCode,  
    string? Message);
```

Допускается использовать enum для статуса, если это соответствует текущим application contracts.

Семантика полей

RunId

Идентификатор созданного `crawler_run`.

Status

Допустимые значения:

ok
error

Не возвращать `running` после завершения метода.

Source

Источник discovery:

```
category-seed  
api  
sitemap
```

При ошибке до определения источника допустимо:

```
discovery
```

DiscoveredCount

Количество URL после завершения `IProductUrlDiscoveryService`.

Это уже результат существующего discovery и существующего `ProductUrlFilter`.

AcceptedCount

Количество валидных элементов, переданных repository после преобразования.

Если repository выполняет дополнительную дедупликацию, использовать его `ReceivedCount` как фактический `AcceptedCount`.

InsertedCount

Количество новых строк каталога.

UpdatedCount

Количество обновлённых строк каталога.

SkippedCount

Рассчитывать детерминированно:

```
DiscoveredCount - AcceptedCount
```

Значение не должно быть отрицательным.

ErrorCode

При успехе:

```
null
```

При ошибке — стабильный код ошибки.

Message

Краткое описание результата или ошибки.

Не помещать stack trace в Message.

3. Добавить RefreshProductCatalogUseCase

Добавить класс:

```
public sealed class RefreshProductCatalogUseCase :  
    IRefreshProductCatalogUseCase
```

Рекомендуемые зависимости:

```
IProductUrlDiscoveryService productUrlDiscoveryService  
IProductCatalogRepository productCatalogRepository  
ICrawlerRunRepository crawlerRunRepository  
ILogger<RefreshProductCatalogUseCase> logger
```

Если текущая модель запусков требует IIngestionRunRepository, не подключать её автоматически.

ingestion_run предназначен для процесса загрузки наблюдений и цен. Обновление каталога не должно создавать ingestion run без отдельного архитектурного обоснования.

В рамках MPC-62 предпочтительно использовать только crawler_run.

Required execution flow

Use case должен выполнять операции в следующем порядке.

Шаг 1. Создать crawler_run

Создать запись запуска **до начала discovery**:

```
var runId = await crawlerRunRepository.StartAsync(  
    "catalog-refresh",  
    ct);
```

Точное значение source может быть:

catalog-refresh

или:

product-catalog-refresh

Предпочтительно:

catalog-refresh

Это принципиальное требование.

В отличие от существующего `RunCrawlerUseCase`, пользователь должен видеть активный запуск в БД уже во время обхода category seeds.

Шаг 2. Запустить discovery

Вызвать:

```
var discovery = await productUrlDiscoveryService
    .DiscoverProductUrlsAsync(ct);
```

Не обходить категории непосредственно из use case.

Не зависеть напрямую от:

- `CategorySeedProductUrlDiscoveryStrategy`;
- `SitemapProductUrlDiscoveryStrategy`;
- `CategoryPageLoader`;
- `CategorySeedProvider`.

Use case должен зависеть от абстракции:

`IProductUrlDiscoveryService`

Шаг 3. Преобразовать URL в catalog upsert items

Каждый URL преобразовать в:

```
ProductCatalogUpsertItem
```

Пример:

```
new ProductCatalogUpsertItem(  
    Source: source,  
    Url: url,  
    NormalizedUrl: url,  
    ExternalId: null,  
    Slug: TryExtractSlug(url),  
    DiscoveredAtUtc: discoveredAt);
```

Source

Для записей `product_catalog.source` использовать стабильное имя магазина:

```
varus
```

Не использовать в качестве product source:

```
category-seed  
sitemap  
api  
catalog-refresh
```

Причина: discovery strategy может измениться, но товар относится к одному источнику данных — Varus.

Источник discovery должен храниться в результате запуска и логах, но не должен менять уникальный ключ товарного каталога.

Если MPC-61 уже определила другое стабильное соглашение для `source`, следовать ему и указать это в отчёте.

4. Использовать единый timestamp batch

Получить время один раз:

```
var discoveredAt = DateTimeOffset.UtcNow;
```

Использовать один `discoveredAt` для всех элементов одного refresh batch.

Не вызывать `DateTimeOffset.UtcNow` отдельно для каждого URL.

Для тестируемости предпочтительно использовать существующую abstraction времени, если она уже есть в проекте.

Не добавлять новую глобальную clock abstraction только ради этой задачи, если в проекте её нет.

5. URL normalization

Не создавать вторую независимую реализацию нормализации.

`IProductUrlDiscoveryService` уже возвращает URL после `ProductUrlFilter`, который вызывает существующую нормализацию.

Поэтому:

```
Url = discovery URL
NormalizedUrl = discovery URL
```

Допускается дополнительный trim.

Запрещено:

- удалять части path;
- менять регистр path;
- декодировать URL вручную;
- удалять trailing slash отдельной новой логикой;
- создавать новый `ProductUrlNormalizer`, дублирующий `ProductUrlFilter.NormalizeProductUrl`.

Если для корректности требуется общий normalizer, выполнить минимальный refactoring:

```
ProductUrlFilter.NormalizeProductUrl
→ IProductUrlNormalizer / ProductUrlNormalizer
```

только если это не меняет существующее поведение и покрыто regression tests.

Предпочтительно не делать этот refactoring в MPC-62.

6. Slug

Slug можно получить из последнего непустого path segment.

Пример:

```
https://varus.ua/product-name  
→ product-name
```

Slug является optional metadata.

Ошибка извлечения slug не должна останавливать refresh.

Для URL без подходящего path segment:

```
Slug = null
```

Не использовать slug как идентификатор или ключ уникальности.

7. Выполнить один batch upsert

Вызвать repository один раз:

```
var upsertResult = await productCatalogRepository  
.UpsertDiscoveredAsync(items, ct);
```

Обязательное требование:

`RefreshProductCatalogUseCase` не должен вызывать repository отдельно для каждого URL.

Запрещено:

```
foreach (var item in items)  
{  
    await repository.UpsertAsync(item, ct);  
}
```

8. Завершить `crawler_run` успешно

При успешном batch upsert:

```
await crawlerRunRepository.FinishAsync(  
    runId,  
    RunStatus.Ok,
```

```
note,  
ct);
```

Пример note:

```
discovery_source=category-seed, discovered=40000, accepted=39980,  
inserted=150, updated=39830, skipped=20
```

Note должна быть:

- краткой;
- агрегированной;
- без полного списка URL;
- без stack trace.

9. Вернуть успешный результат

Пример:

```
new RefreshProductCatalogResult(  
    RunId: runId,  
    Status: "ok",  
    Source: "category-seed",  
    DiscoveredCount: 40000,  
    AcceptedCount: 39980,  
    InsertedCount: 150,  
    UpdatedCount: 39830,  
    SkippedCount: 20,  
    ErrorCode: null,  
    Message: note);
```

Error handling

1. Discovery unavailable

Перехватить:

```
ProductUrlDiscoveryUnavailableException
```

Использовать error code:

```
product_url_discovery_unavailable
```

или существующую константу:

```
CrawlerErrorCodes.ProductUrlDiscoveryUnavailable
```

Завершить `crawler_run` со статусом:

```
error
```

Вернуть result:

```
DiscoveredCount = 0
AcceptedCount = 0
InsertedCount = 0
UpdatedCount = 0
SkippedCount = 0
```

2. Unexpected discovery error

Для необработанного исключения discovery использовать стабильный код:

```
catalog_discovery_failed
```

Не возвращать тип исключения как error code.

3. Catalog persistence error

Если discovery завершилось, но `UpsertDiscoveredAsync` завершился исключением:

```
catalog_upsert_failed
```

`crawler_run` должен быть завершён со статусом `error`.

В note/message указать:

- discovery source;
- discovered count;
- краткое сообщение ошибки.

4. Cancellation

При:

```
OperationCanceledException
```

если `ct.IsCancellationRequested == true`:

1. попытаться завершить `crawler_run` со статусом `error`;
2. использовать note:

```
catalog refresh cancelled
```

1. повторно выбросить `OperationCanceledException`.

Не превращать cancellation в обычный успешный result.

Если завершить run невозможно из-за отменённого token, использовать безопасный cleanup token только если такой подход уже принят в проекте.

Не создавать бесконечный или длительный cleanup.

5. Failure while finishing run

Если основная операция уже завершилась ошибкой, а `crawlerRunRepository.FinishAsync` тоже упал:

- не скрывать исходную ошибку;
- записать вторичную ошибку в logger;
- сохранить исходный error code;
- не возвращать ложный успех.

Если завершение run упало после успешного upsert, метод должен считаться ошибочным, поскольку состояние запуска в БД осталось некорректным.

Empty discovery behavior

Существующий `IProductUrlDiscoveryService` при отсутствии URL должен бросать:

```
ProductUrlDiscoveryUnavailableException
```

Use case не должен интерпретировать пустой каталог как успешный refresh.

Причина: пустой результат может означать:

- изменение HTML Varus;
- поломку link extractor;
- блокировку запросов;
- неверную конфигурацию;
- пустой seed file.

Результат должен быть:

```
status=error
```

Logging

Добавить структурированные логи.

Начало

```
Product catalog refresh started. RunId={RunId}
```

Discovery завершён

```
Product catalog discovery completed. RunId={RunId};  
DiscoverySource={DiscoverySource}; Discovered={Discovered}
```

Upsert завершён

```
Product catalog refresh completed. RunId={RunId};  
DiscoverySource={DiscoverySource}; Discovered={Discovered};  
Accepted={Accepted}; Inserted={Inserted}; Updated={Updated};  
Skipped={Skipped}
```

Ошибка

```
Product catalog refresh failed. RunId={RunId}; ErrorCode={ErrorCode};  
DiscoverySource={DiscoverySource}; Discovered={Discovered}
```

Исключение передавать logger отдельным параметром.

Не логировать каждый URL на уровне Information.

Не логировать весь batch.

DI registration

Зарегистрировать use case в Application DI:

```
services.AddScoped<RefreshProductCatalogUseCase>();  
services.AddScoped<IRefreshProductCatalogUseCase>(  
    provider => provider.GetRequiredService<RefreshProductCatalogUseCase>());
```

Допускается сокращённая регистрация:

```
services.AddScoped<IRefreshProductCatalogUseCase,  
    RefreshProductCatalogUseCase>();
```

Следовать существующему стилю проекта.

Runtime entry point

В рамках MPC-62 use case должен быть доступен для ручного запуска.

Предпочтительный минимальный вариант — добавить отдельный аргумент Worker:

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

При запуске без аргументов существующее поведение Worker не должно измениться.

Required behavior

```
catalog-refresh  
→ выполнить только RefreshProductCatalogUseCase  
→ завершить процесс
```

Не запускать:

- `RunCrawlerUseCase`;
- price extraction;
- queue drain;
- price snapshot collection.

Invalid command

Для неизвестного аргумента:

- вывести понятное сообщение;
- вернуть ненулевой exit code;
- не запускать crawler автоматически.

Existing default behavior

Если Worker сейчас запускает старый crawler без аргументов, сохранить это поведение для обратной совместимости.

Полное разделение режимов Worker будет выполнено в следующем тикете. В MPC-62 требуется только минимальный ручной entry point для проверки use case.

Configuration

Не добавлять новые настройки, если они не нужны.

Use case должен использовать существующие:

```
"Crawler": {  
  "DiscoveryMode": "CategorySeeds",  
  "CategorySeedUrlsFilePath": "...",  
  "VegetablesUrlContains": "",  
  "MaxUrls": 20000,  
  "MaxCategoryPagesPerSeed": 10  
}
```

Important

Для полного refresh каталога:

```
"VegetablesUrlContains": ""
```

Но use case не должен программно переопределять эту настройку.

Если фильтр настроен пользователем, use case обязан уважать его.

Не изменять configuration во время runtime.

MaxProductsPerRun semantics

`MaxProductsPerRun` относится к price collection и не должен ограничивать catalog refresh.

Однако в текущем коде `ProductUrlFilter` вычисляет лимит так:

```
min(MaxProductsPerRun, MaxUrls)
```

Это делает невозможным сохранение полного каталога при:

```
"MaxProductsPerRun": 200
```

Поэтому в рамках MPC-62 необходимо исправить разделение лимитов.

Required change

Для catalog discovery итоговый лимит должен определяться только:

```
MaxUrls
```

Для price collection лимит должен продолжать определяться:

```
MaxProductsPerRun
```

Нельзя просто увеличить `MaxProductsPerRun` до 50000 в конфигурации как обход проблемы.

Required limit refactoring

Устранить смешение двух разных ограничений:

```
Discovery limit  
Price collection limit
```

Предпочтительный вариант

Изменить `IProductUrlFilter`:

```
IReadOnlyList<string> Apply(  
    IEnumerable<Uri> urls,
```

```
string sourceName,  
int maxResults);
```

В `ProductUrlDiscoveryService` передавать:

```
crawlerOptions.Value.MaxUrls
```

Существующий `RunCrawlerUseCase` после discovery должен самостоятельно ограничивать список:

```
discovery.Url  
    .Take(Math.Max(1, options.Value.MaxProductsPerRun))
```

до формирования `QueueEnqueueItem`.

Required resulting semantics

Catalog refresh

```
Discovery → максимум MaxUrls  
Upsert → максимум MaxUrls
```

Existing price crawler

```
Discovery → максимум MaxUrls  
Queue → максимум MaxProductsPerRun
```

Example

```
"MaxUrls": 50000,  
"MaxProductsPerRun": 200
```

Результат:

```
RefreshProductCatalogUseCase:  
до 50000 URL в product_catalog  
  
RunCrawlerUseCase:  
до 200 URL в price_collect_queue
```

Это обязательная часть MPC-62.

Backward compatibility

После изменения лимитов:

- текущий crawler всё ещё должен обрабатывать максимум `MaxProductsPerRun` ;
- discovery может вернуть больше элементов;
- queue не должна получить больше `MaxProductsPerRun` ;
- существующие тесты должны быть скорректированы;
- поведение `VegetablesUrlContains` не должно измениться;
- поведение exclusions из `url-filters.json` не должно измениться;
- URL normalization не должна измениться.

Tests

Добавить unit tests для нового use case и regression tests для разделения лимитов.

Required use case tests

1. Successful refresh

`ExecuteAsync_Success_UpsertsDiscoveredUrlsAndFinishesRun`

Проверить:

- `crawler_run.StartAsync` вызывается до discovery;
- discovery вызывается один раз;
- repository вызывается один раз;
- в repository переданы все URL;
- `crawler_run.FinishAsync` получает `RunStatus.Ok` ;
- result содержит корректные counters.

2. Run exists during discovery

`ExecuteAsync_StartsCrawlerRunBeforeDiscovery`

Проверить порядок вызовов:

`StartAsync`
`DiscoverProductUrlsAsync`
`UpsertDiscoveredAsync`
`FinishAsync`

Порядок является обязательным.

3. Stable product source

ExecuteAsync_UsesVarusAsProductCatalogSource

Проверить, что catalog items получают:

Source = "varus"

а не:

category-seed

4. Discovery source in result

ExecuteAsync_ReturnsActualDiscoverySource

Проверить mapping для:

- CategorySeed;
- Sitemap;
- Api.

5. One timestamp per batch

ExecuteAsync_UsesSameDiscoveredAtForAllItems

Все `ProductCatalogUpsertItem.DiscoveredAtUtc` должны быть одинаковыми.

6. Slug extraction

ExecuteAsync_ExtractsSlugFromProductUrl

Проверить:

`https://varus.ua/product-name`
→ product-name

7. Batch repository call

```
ExecuteAsync_MultipleUrls_CallsRepositoryOnce
```

8. Upsert counters

```
ExecuteAsync_ReturnsRepositoryCounters
```

Fake repository возвращает:

```
Received=100  
Inserted=10  
Updated=90
```

Result должен содержать соответствующие значения.

9. Discovery unavailable

```
ExecuteAsync_DiscoveryUnavailable_FinishesRunWithError
```

Проверить:

- run создан;
- finish вызван с `RunStatus.Error`;
- repository не вызван;
- error code корректный;
- counters равны нулю.

10. Discovery exception

```
ExecuteAsync_DiscoveryThrowsUnexpectedException_FinishesRunWithError
```

11. Repository exception

```
ExecuteAsync_RepositoryThrows_FinishesRunWithError
```

Проверить:

- discovered count сохранён в result/message;

- run завершён с error;
- ложный success не возвращается.

12. Cancellation

ExecuteAsync_Cancelled_RethrowsOperationCanceledException

Проверить:

- cancellation не преобразуется в `ok`;
- выполняется попытка завершить run.

Required ProductUrlFilter regression tests

13. Filter respects explicit max result count

Apply_WhenMaxResultsProvided_ReturnsAtMostRequestedCount

14. MaxUrls no longer depends on MaxProductsPerRun

DiscoverProductUrlsAsync_MaxProductsPerRunIs200AndMaxUrlsIs50000_ReturnsMoreThan200Urls

Использовать synthetic source с числом URL больше 200.

15. Existing crawler still limits queue

RunVegetablesAsync_DiscoveryReturnsMoreThanMaxProductsPerRun_EnqueuesOnlyMaxProductsPerRun

Пример:

discovery returns 500
MaxProductsPerRun = 200
queue receives 200

16. MaxUrls limits catalog refresh

RefreshProductCatalog_MaxUrlsIs1000_UpsertsAtMost1000Items

17. Empty vegetable filter

ProductUrlFilter_EmptyVegetablesFilter_DoesNotExcludeValidUrls

18. Vegetable filter remains supported

ProductUrlFilter_ConfiguredVegetablesFilter_ExcludesNonMatchingUrls

Integration tests

Добавить integration test с реальной PostgreSQL test database.

1. Full catalog refresh insert

RefreshProductCatalogUseCase_NewUrls_PersistsCatalogItems

Проверить:

- создан `crawler_run`;
- run завершён `ok`;
- записи появились в `product_catalog`;
- количество соответствует discovery;
- `source = varus`.

2. Repeat refresh update

RefreshProductCatalogUseCase_RepeatedUrls_UpdateExistingCatalogRows

Проверить:

- количество строк не удваивается;
- `first_discovered_at` сохраняется;
- `last_discovered_at` обновляется;
- run завершён успешно.

3. Run created before long discovery

Если integration fixture позволяет управлять fake discovery, проверить наличие `crawler_run` со статусом running до завершения discovery.

Если такой тест технически чрезмерен, покрыть порядок unit test и явно указать это в отчёте.

4. No price data created

RefreshProductCatalogUseCase_DoesNotCreatePriceCollectionData

Проверить, что после catalog refresh не создаются новые строки в:

- `price_collect_queue`;
 - `price_snapshot`;
 - `product`, если product создаётся только через observation flow;
 - `ingestion_run`.
-

Documentation

Обновить:

docs/architecture.md

Добавить отдельный flow:

```
Catalog refresh
  ↓
IProductUrlDiscoveryService
  ↓
ProductUrlFilter
  ↓
RefreshProductCatalogUseCase
  ↓
IProductCatalogRepository
  ↓
product_catalog
```

Явно описать:

- `product_catalog` теперь подключён к runtime;
- catalog refresh не собирает цены;
- catalog refresh использует `MaxUrls`;

- price collection использует `MaxProductsPerRun` ;
- source товара остаётся `varus` ;
- discovery source хранится в статистике запуска;
- `crawler_run` создаётся до discovery.

Обновить README с примером ручного запуска:

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

Non-goals

В рамках MPC-62 запрещено реализовывать:

- выбор товаров из `product_catalog` для сбора цен;
- `CollectProductPricesUseCase` ;
- oldest-first scheduling;
- обновление `last_checked_at` ;
- обновление `next_check_at` ;
- error backoff для catalog items;
- деактивацию отсутствующих товаров;
- `is_active = false` для исчезнувших товаров;
- полный scheduler;
- cron;
- daily hosted service;
- потоковый enqueue во время discovery;
- параллельный запуск discovery и price collection;
- изменение `price_collect_queue` schema;
- UI каталога;
- Web API endpoints каталога;
- удаление существующего `RunCrawlerUseCase` ;
- переименование существующих таблиц;
- новый discovery strategy;
- изменение HTML parser;
- изменение Varus extractor;
- получение `external_id` из карточки товара;
- обработку всех 40 000 карточек;
- оптимизацию производительности category discovery вне необходимого scope.

Important design decisions

1. `crawler_run` создаётся до discovery

Это устраняет текущую проблему наблюдаемости, когда обход сотен category seeds уже выполняется, но в БД ещё нет записи запуска.

2. Catalog source и discovery source различаются

```
Catalog source: varus
Discovery source: category-seed / sitemap / api
```

Их нельзя смешивать.

3. MaxUrls и MaxProductsPerRun разделяются

```
MaxUrls
→ ограничивает размер discovery/catalog refresh

MaxProductsPerRun
→ ограничивает число карточек для price collection
```

4. Use case выполняет один batch upsert

Количество URL не должно превращаться в такое же количество repository calls.

5. Catalog refresh не является price ingestion

Не создавать `ingestion_run`, queue items или snapshots.

Acceptance criteria

Задача считается выполненной, если:

- [] добавлен `IRefreshProductCatalogUseCase` ;
- [] добавлен `RefreshProductCatalogUseCase` ;
- [] добавлен `RefreshProductCatalogResult` ;
- [] use case создаёт `crawler_run` до discovery;
- [] discovery вызывается через `IProductUrlDiscoveryService` ;
- [] найденные URL преобразуются в `ProductCatalogUpsertItem` ;
- [] catalog source имеет стабильное значение `varus` ;
- [] discovery source корректно отображается в result и logs;
- [] один batch использует один `DiscoveredAtUtc` ;
- [] repository вызывается один раз на batch;
- [] успешный запуск завершает `crawler_run` со статусом `ok` ;
- [] ошибка discovery завершает run со статусом `error` ;
- [] ошибка upsert завершает run со статусом `error` ;
- [] cancellation не превращается в успешный result;
- [] empty discovery считается ошибкой;
- [] use case зарегистрирован в DI;
- [] Worker поддерживает ручную команду `catalog-refresh` ;

- [] команда `catalog-refresh` не запускает сбор цен;
- [] `MaxUrls` ограничивает discovery/catalog refresh;
- [] `MaxProductsPerRun` ограничивает только price queue;
- [] существующий crawler при лимите 200 ставит в очередь максимум 200 товаров;
- [] catalog refresh при `MaxUrls > 200` способен сохранить больше 200 URL;
- [] не создаются `price_snapshot` и queue items;
- [] добавлены unit tests;
- [] добавлены regression tests разделения лимитов;
- [] добавлены PostgreSQL integration tests;
- [] обновлена архитектурная документация;
- [] обновлён README;
- [] существующие тесты проходят;
- [] solution собирается без новых warnings;
- [] unrelated code не рефакторился.

Required verification

Codex обязан выполнить:

```
dotnet restore
dotnet build --no-restore
dotnet test --no-build
```

Для integration tests использовать существующую PostgreSQL test infrastructure проекта.

Выполнить ручную проверку:

```
dotnet run --project VarPrice.Worker -- catalog-refresh
```

После запуска проверить:

```
select
    id,
    source,
    started_at,
    finished_at,
    status,
    note
from crawler_run
order by id desc
limit 5;
```

Проверить каталог:

```
select count(*)
from product_catalog;
```

Проверить распределение:

```
select
    source,
    count(*)
from product_catalog
group by source;
```

Проверить отсутствие дубликатов:

```
select
    source,
    normalized_url,
    count(*)
from product_catalog
group by source, normalized_url
having count(*) > 1;
```

Запрос не должен возвращать строк.

Проверить, что catalog refresh не создавал price queue:

```
select count(*)
from price_collect_queue
where run_id = :run_id;
```

Ожидается:

0

Manual scenario

Использовать настройки:

```
{
  "Crawler": {
    "DiscoveryMode": "CategorySeeds",
    "VegetablesUrlContains": "",
    "MaxUrls": 50000,
```

```
"MaxProductsPerRun": 200,  
"MaxCategoryPagesPerSeed": 10  
}  
}
```

Ожидаемое поведение:

```
catalog-refresh:  
может сохранить больше 200 URL,  
но не больше 50000  
  
обычный crawler:  
может поставить в price queue не больше 200 URL
```

Это поведение обязательно подтвердить тестом или ручной проверкой.

Codex final response format

В финальном ответе Codex обязан предоставить следующие разделы.

1. Summary

Кратко описать реализованный catalog refresh flow.

2. Changed files

Перечислить все изменённые и добавленные файлы с назначением.

3. Runtime flow

Показать фактический порядок:

```
crawler_run_start  
→ discovery  
→ batch catalog upsert  
→ crawler_run_finish
```

4. Limits behavior

Явно подтвердить:

```
MaxUrls controls discovery/catalog refresh
MaxProductsPerRun controls price collection
```

Указать тест, подтверждающий разделение.

5. Database behavior

Указать:

- сколько catalog repository calls выполняется на batch;
- создаётся ли `ingestion_run`;
- создаются ли queue items;
- создаются ли snapshots.

6. Tests

Указать:

- добавленные unit tests;
- добавленные integration tests;
- фактический результат `dotnet build`;
- фактический результат `dotnet test`;
- результат ручного `catalog-refresh`.

7. Risks

Указать известные ограничения:

- catalog refresh пока не деактивирует исчезнувшие товары;
- price collection пока не читает `product_catalog`;
- полный scheduler будет реализован отдельно.

8. Verification evidence

Привести фактические агрегированные результаты ручного запуска:

```
RunId:
DiscoverySource:
Discovered:
Accepted:
Inserted:
Updated:
Skipped:
Catalog row count:
```

Не заявлять об успешной проверке, если команды фактически не выполнялись.